

Systolic Array Architecture and Baseline Analysis

Test setup and verified results

The baseline was tested on the Terasic DE25 Standard Agilinx 5 board using the JTAG Master Bridge through Quartus System Console. The test script wrote operands, armed input and output DMA, started the array, and compared the returned matrix with a reference. These results do not measure Linux HPS application performance. JTAG transfer timing is outside this report's scope.

The first implementation used an unoptimized 4 by 4 systolic array at 50 MHz. Its hardware performance counter recorded 44 cycles, and all 16 output elements matched the reference with zero errors.

Metric	Baseline value
FPGA device	Intel Agilinx 5 A5ED013BB32AE4SCS
Development board	Terasic DE25 Standard
Test interface	JTAG Master Bridge through System Console
Array size	4 by 4 with 16 processing elements
Arithmetic	Signed INT8 operands and INT32 accumulators
Clock	50 MHz with a 20 ns period
Matrix result	PASS with all 16 elements correct
Reported error count	0
Hardware counter	44 cycles
Counter interval	0.880 microseconds at 50 MHz
Work per tile	64 multiply accumulate operations or 128 operations
Input payload	32 bytes in 8 packed words
Output payload	64 bytes in 16 words

Baseline architecture

The 4 by 4 design has one operand bank and one result snapshot. Each tile loads, computes, and captures sequentially. A continuous mode can start the next buffered tile after completion, but it does not overlap these stages. Every tile resets the accumulators. Hardware K accumulation, overlapping double buffering, and a host tiling engine were not part of this baseline.

The 44 cycle counter includes loading and capture after execution starts. It excludes earlier JTAG and DMA setup and does not wait for output DMA to finish writing its destination. This timing boundary matters when comparing the baseline with the optimized design.

Theoretical calculations

A multiply accumulate operation consists of one multiplication and one addition, so it counts as two operations. A full 4 by 4 multiplication performs 4 times 4 times 4, or 64 MACs, equivalent to 128 operations.

Quantity	Calculation	Result
Clock period	1 divided by 50 million clocks per second	20 ns
Peak MAC rate	16 PEs times 50 MHz	800 MMAC per second
Peak compute rate	16 PEs times 50 MHz times 2 operations	1.600 GOPS
Tile counter time	44 cycles times 20 ns	0.880 microseconds
Useful MAC rate	64 MACs divided by 0.880 microseconds	72.727 MMAC per second
Useful compute rate	128 operations divided by 0.880 microseconds	0.14545 GOPS
Useful rate divided by peak	0.14545 divided by 1.600	9.09 percent
Arithmetic intensity	128 operations divided by 96 payload bytes	1.333 operations per byte

Peak assumes every PE performs useful work every clock. The 9.09 percent figure compares useful work over the full counter interval with that peak; it is not a directly measured per cycle PE activity percentage.

Cycle allocation in the 4 by 4 design

Controller state	Counted cycles
Load matrix A	4
Load matrix B	4
Reset PE accumulators	2
Feed skewed operands	7
Flush and settle	8
Capture and registered handshakes	19
Total performance counter	44
DONE state	One additional clock not counted

Capture takes 19 clocks. The DONE state adds one clock outside the performance counter. With no stalls and the next input already buffered, sequential continuous execution takes 45 clocks per tile: 0.900 microseconds and an ideal useful rate of 0.14222 GOPS.

Payload rate

Dividing 96 payload bytes by 0.880 microseconds gives 109.09 MB per second, or 104.04 MiB per second. This is a payload divided by counter time calculation, not measured DMA or JTAG bandwidth. Input and output transfers are not fully timed by that counter.

Resources storage and timing

DSP implementation and scaling

Scaling the array from 4 by 4 to 16 by 16 under the baseline mapping requires 256 physical DSP blocks, exceeding the device capacity of 188. The 16 by 16 implementation therefore uses ALM based arithmetic. The 4 by 4 implementation uses 16 physical DSP blocks in Independent 18x18 Plus 36 mode, consistent with one DSP per processing element. Both standalone and integrated 16 by 16 fitter summaries report zero DSP blocks.

DSP budget using the 4 by 4 mapping

Quantity	Calculation	Result
Physical DSP capacity	Device capacity and baseline fitter results	188 blocks
4 by 4 DSP utilization	16 PEs at one physical DSP per PE	16 blocks or 8.51 percent
Projected 16 by 16 DSP use	256 PEs at the same mapping	256 blocks or 136.17 percent
Projected capacity shortfall	256 minus 188	68 blocks
Largest square array under this mapping	13 squared fits but 14 squared exceeds 188	13 by 13 before other DSP use
16 by 16 ALM implementation	Standalone and integrated fitter summaries	Zero DSP blocks

The DSP estimate applies the baseline mapping of one physical DSP per processing element to a 16 by 16 array. The resulting requirement exceeds capacity by 68 blocks. Alternative DSP packing modes may reduce this requirement; the estimate applies specifically to the baseline mapping.

The device has 188 variable precision DSP blocks and 376 18x19 multiplier resources. These are different units. Physical DSP utilization is 16 divided by 188, or 8.51 percent.

Capacity reference: Agilx 5 E Series product table, <https://cdrdv2-public.intel.com/743772/agilex-5-fpgas-and-socs-e-series-product-table.pdf>. Mode reference: Agilx 5 Variable Precision DSP Blocks User Guide, <https://docs.altera.com/r/docs/813968/current>.

The following table gives the baseline implementation resource utilization. These fitted values include storage and implementation overhead beyond the logical datapath.

Resource	Baseline utilization
Adaptive logic modules	1807
Dedicated logic registers	2817
DSP blocks	16
M20K memory blocks	11
Block memory bits	44416
Pins	6
PLLs	0

Two implementation builds report different logic utilization: 1847 ALMs and 2845 registers on September 1, and 1807 ALMs and 2817 registers on September 29. The table uses the September 29 values. Both builds use 16 DSP blocks. Integrated GHRD totals include HPS interface logic and require separate accounting when compared with accelerator only resource utilization.

Local storage and FIFO capacity

Storage	Payload capacity
Local A and B registers	32 bytes in one operand bank
Result snapshot	64 bytes in one capture bank
PE accumulator registers	64 bytes across 16 PEs
Default input FIFO	32 words or 128 bytes or four input tiles
Default output FIFO	64 words or 256 bytes or four output tiles

The FIFO status value 0x40200000 indicates an input depth of 32 words, an output depth of 64 words, and empty levels. Logical payload capacity differs from fitted block memory utilization because the complete implementation includes additional storage and overhead.

Timing closure

Timing metric	Measured value
Clock period	20.000 ns
Setup slack	Positive 4.952 ns
Hold slack	Positive 0.068 ns
Recovery slack	Positive 17.934 ns
Removal slack	Positive 0.157 ns
Minimum pulse width slack	Positive 8.912 ns

Subtracting the 4.952 ns setup slack from the 20.000 ns clock period gives 15.048 ns, corresponding to an approximate frequency of 66.45 MHz. This slack based estimate does not establish the maximum operating frequency. All baseline performance calculations use the tested 50 MHz clock.

Design changes from the 4 by 4 baseline

Phase 1 Tiling and Block Accumulation CFG_ACCUMULATE

The baseline resets its accumulators for each independent 4 by 4 operation. The 16 by 16 implementation retains partial sums across K tiles using CFG_ACCUMULATE and captures the result after the final step. The Linux HPS driver decomposes larger matrices into tiles; signed accumulation and irregular matrix shapes have been validated.

Phase 2 Partial Tiles and Runtime Dimensions ACT_M ACT_K ACT_N

Runtime dimension registers configure the active matrix dimensions. The Linux HPS driver supports partial edge tiles by zero padding operands to 16 by 16 and retaining only valid output elements. Current tests validate this

padded execution path; native rectangular execution using the active dimension registers has a known feed timing limitation.

Phase 3 Double Buffering Ping Pong Matrix Registers

The baseline uses one operand bank and one result snapshot. Ping pong matrix registers provide separate operand banks so the next tile can be loaded while another bank supplies computation. Double buffering is present in the RTL; the current sequential Linux HPS benchmark does not establish its overlap benefit.

Phase 4 Pipelining and Concurrent Load Compute Drain

The updated architecture supports overlap between operand loading, computation, and result draining. This extends the baseline’s sequential load, compute, and capture sequence. The current host driver submits tile steps sequentially, so the measured wall times do not demonstrate full concurrent pipeline utilization.

Comparison with the optimized design

Comparable architecture and compute figures

Metric	4 by 4 baseline	16 by 16 implementation
Processing elements	16	256
Operating clock used in calculations	50 MHz	100 MHz
Peak throughput	1.600 GOPS	51.200 GOPS
Work in one full tile	128 operations	8192 operations
Full tile input and output	96 bytes	1536 bytes
Full tile arithmetic intensity	1.333 operations per byte	5.333 operations per byte
Useful counter throughput	0.14545 GOPS	About 12.95 GOPS for the 128 cubed run
Hardware accumulation across K	Not supported	Validated
Overlapping double buffering	Not supported	Present in RTL
Arithmetic implementation	DSP based	ALM based with zero DSPs in 16 by 16 fit summaries
Test interface	JTAG Master Bridge	Linux HPS and reserved DDR

The optimized array has 16 times as many PEs and twice the clock, so its theoretical peak is 32 times higher. It performs 64 times the work in a full tile while transferring 16 times the payload, giving four times the full tile arithmetic intensity. These are architecture calculations, not measured end to end speedups.

The counter throughput ratio is approximately 89 times, but the timing boundaries differ: the baseline counter includes load and capture, while the 16 by 16 counter excludes them. The Linux HPS driver submits tiles sequentially, so its benchmark does not fully exercise the RTL overlap capability. At 50 MHz, a 16 by 16 array has a theoretical peak of 25.6 GOPS.

Analytical scaling estimates

The 4 by 4 array does not implement hardware tiling or accumulation across K tiles. The following analytical estimates assume software decomposition into independent 4 by 4 invocations, with partial results accumulated on the CPU.

Matrix workload	Independent invocations	Sum of 44 cycle intervals at 50 MHz
64 by 64 times 64 by 64	4096	3.60448 ms
128 by 128 times 128 by 128	32768	28.83584 ms

Invocation count equals M divided by 4, times N divided by 4, times K divided by 4 for these divisible dimensions. Payload would be 393216 bytes and 3145728 bytes respectively if each invocation transfers fresh operands and its partial result. These figures omit DONE and launch gaps, transfer overhead, stalls, and CPU accumulation. They are not measured baseline application runtimes.

Linux HPS wall time

Linux HPS wall time is measured with a chrono timer around run_tiled_gemm. The interval includes operand packing, resets, buffer initialization, CSR and DMA submission, completion waits, and output decoding. Linux boot, driver initialization, memory mapping, CPU reference execution, and result verification are excluded. This elapsed time is independent of the hardware performance counter.

Measured run	CPU reference ms	FPGA host wall time ms
64 cubed with 10 microsecond polling sleep	1.11307	12.02661
64 cubed with busy polling	1.11362	9.10889
128 cubed with busy polling	3.36169	45.41792

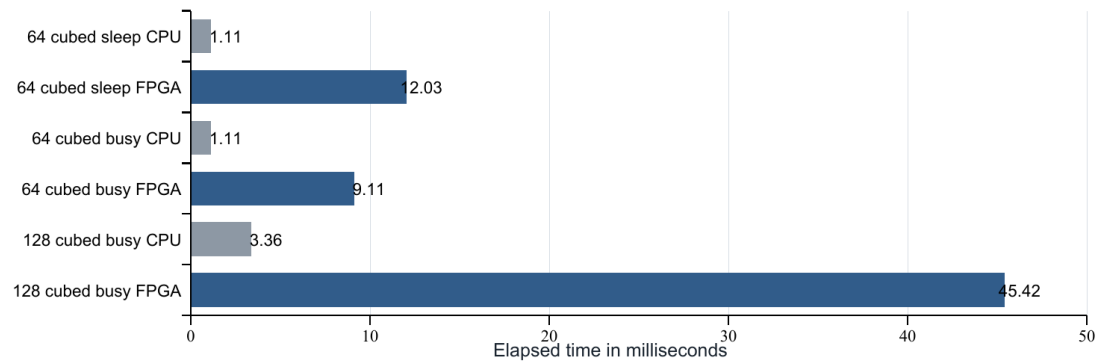
The table contains individual board measurements. Repeated measurements and latency distributions for the 16 by 16 implementation are presented in the separate GEMM validation and scaling report. No Linux HPS wall time was measured for the 4 by 4 baseline, so its counter based estimates cannot establish an application speedup.

Linux HPS performance of the 16 by 16 array

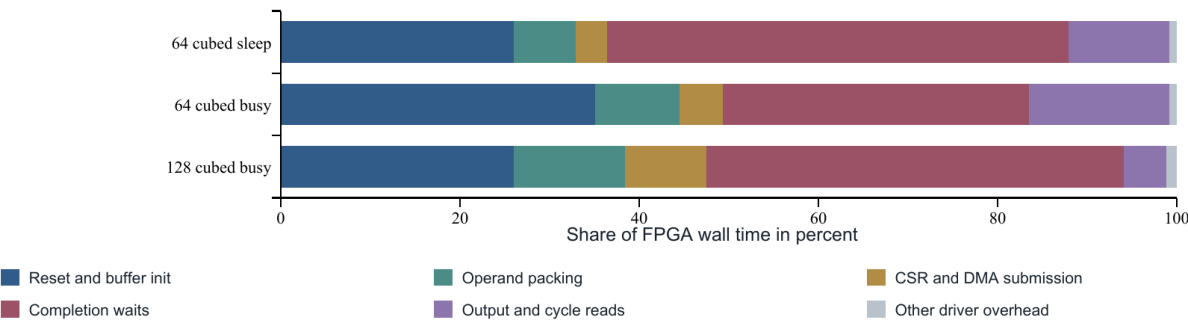
Measured Linux HPS execution time

New 16x16 design | Board runs on October 1 2026 | All outputs verified

A CPU reference and FPGA end-to-end time



B Where the FPGA end-to-end time was spent



Single observations, not averages. Completion waits include compute, DMA, and polling.

Figure 1 presents three Linux HPS measurements. Panel A compares scalar CPU reference time with the complete FPGA driver interval. Panel B shows the measured timing stages as fractions of wall time; other overhead is the residual after subtracting the five reported stages. All three runs passed with zero mismatches. Each bar represents one observation.

The plot characterizes the 16 by 16 implementation. Completion waits and reset or buffer initialization are the largest costs in these measurements. Busy polling reduced the measured 64 cubed wall time by 24.3 percent. The scalar CPU reference completed faster than the FPGA driver in all three observations. No baseline HPS measurement is available for an application level comparison between the two array sizes.